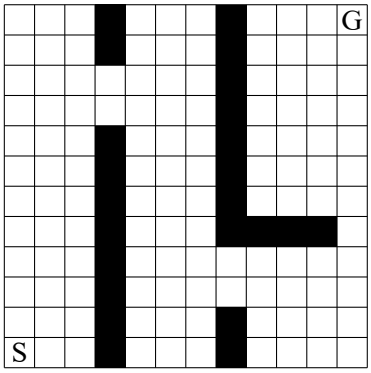
3.4. Xây dựng kịch bản mô phỏng và đánh giá so sánh đa thuật toán

3.4.1. Thiết lập môi trường mô phỏng trên nền tảng Colab: Bản đồ lưới(Grid Map)

Google Colab được lựa chọn để xây dựng mô phỏng bằng Python vì cho phép chạy notebook trên trình duyệt, thuận tiện lưu mã nguồn, hiển thị đường đi và chia sẻ cấu hình thí nghiệm [1]. NumPy dùng để biểu diễn bản đồ và xử lý tọa độ; Matplotlib dùng để vẽ lưới và kết quả; mô-đun `heapq` hỗ trợ hàng đợi ưu tiên cho Dijkstra và A*. Mô phỏng sử dụng CPU, không yêu cầu GPU; phiên bản thư viện và cấu hình phiên chạy cần được lưu cùng dữ liệu.

Môi trường được biểu diễn bằng ma trận nhị phân M có kích thước $H \times W$, gồm các ô vuông cạnh Δ mét, bố trí như ô bàn cờ. Quy ước **ô trắng là ô có thể đi**, tương ứng $M_{ij} = 0$; **ô đen là ô tường hoặc vật cản**, tương ứng $M_{ij} = 1$. Các ô không bắt buộc xen kẽ màu. Điểm xuất phát S và điểm đích G đều thuộc ô trắng. Với gốc tọa độ tại góc dưới trái, tâm ô cột j, hàng i, đánh số từ 0, được quy đổi thành $(x, y) = ((j + 1/2)\Delta, (i + 1/2)\Delta)$.



Hình 3.8: Minh họa Grid Map: ô trắng có thể đi, ô đen là tường

Dijkstra và A* dùng lưới 8 hướng, với chi phí Δ cho bước ngang/dọc và $\sqrt{2}\Delta$ cho bước chéo; cấm đi chéo xuyên góc tường. PRM, RRT* và APF dùng cùng bản đồ hình học; mọi đoạn nối đều phải được kiểm tra va chạm. Vật cản được mở rộng theo bán kính bao thân xe cộng khoảng dự phòng trước khi lập kế hoạch, để đường của tâm xe không đi quá sát tường.

Ba nhóm kịch bản gồm: không vật cản, mê cung tĩnh và mật độ vật cản cao. Trong từng kịch bản, giữ nguyên bản đồ, độ phân giải và cặp S, G cho năm thuật toán. PRM và RRT* được chạy lặp với các hạt giống ngẫu nhiên đã lưu; mọi phương pháp dùng cùng thời hạn xử lý và điều kiện đạt đích. Đây là thiết kế thí nghiệm; các chỉ số chỉ được điền sau khi thực sự chạy mô phỏng.

3.4.2. So sánh Dijkstra, A*, PRM, RRT* và APF theo bốn tiêu chí đánh giá

Bảng 3.1 đối chiếu đặc tính dự kiến theo bốn tiêu chí đã xác định. Các nhận xét là cơ sở lựa chọn phương pháp và tổ chức phép đo, chưa phải kết quả định lượng của Colab [3, 6, 7].

Bảng 3.1: So sánh các thuật toán trên cùng Grid Map

Thuật toán	**Chiều dài đường đi (m)**	**Thời gian CPU (ms)**	**Độ trơn:** $\int \kappa^2 ds$	**Thành công tránh va chạm (%)**
Dijkstra	Ngắn nhất trên đồ thị nếu trọng số là độ dài.	Có thể mở rộng nhiều ô; phụ thuộc kích thước lưới.	Đường gấp khúc; cần làm mượt trước khi tính độ cong.	Tìm được đường nếu tồn tại trên lưới và đủ thời gian.
A*	Cùng chi phí tối ưu với Dijkstra khi heuristic phù hợp.	Heuristic tốt thường giảm số ô mở rộng.	Vẫn có góc gãy; không tự bảo đảm đường trơn.	Đầy đủ trên lưới hữu hạn với cài đặt đúng và đủ thời gian.
PRM	Ngắn nhất trên roadmap khi dùng Dijkstra; chưa tối ưu toàn miền.	Tốn công xây roadmap; có thể dùng lại cho nhiều truy vấn.	Các cạnh thẳng tạo góc tại nút nối.	Phụ thuộc số mẫu, kết nối và độ hẹp hành lang.
RRT*	Cải thiện nhờ nối lại cây; tối ưu tiệm cận dưới các giả thiết.	Phụ thuộc số mẫu; thêm chi phí tìm lân cận và nối lại.	Bộ nối thẳng vẫn tạo góc; cần làm mượt.	Đầy đủ xác suất dưới các giả thiết; hữu hạn mẫu vẫn có thể thất bại.
APF	Không bảo đảm ngắn nhất; phụ thuộc trường thế và tham số.	Mỗi bước thường đơn giản; mắc kẹt có thể làm hết thời hạn.	Có thể đổi hướng êm, nhưng vẫn dao động gần tường.	Có thể mắc cực tiểu cục bộ; không bảo đảm tới đích.

Cách lượng hóa các tiêu chí. Với $\mathbf{p}_i$ là điểm đường đi theo đơn vị mét và n đoạn nối, chiều dài được tính bằng

$$L = \sum_{i=0}^{n-1} \|\mathbf{p}_{i+1} - \mathbf{p}_i\|.$$

Thời gian CPU đo bằng `time.process_time()`, đổi sang ms, không gồm vẽ hình và đọc tệp. PRM phải báo cáo riêng thời gian xây roadmap và truy vấn; nhiệm vụ mới tính cả hai. Các lần đo dùng cùng phiên Colab; tốc độ của board phải được đo riêng.

Độ trơn $J_\kappa = \int_0^L \kappa(s)^2\, ds$ càng nhỏ càng ít tích lũy độ cong. Đường gấp khúc không khả vi tại góc, nên chỉ tính chỉ số sau cùng một quy trình làm mượt, lấy mẫu và kiểm tra va chạm lại; lưu cả chiều dài trước/sau xử lý. Chỉ số này không thay thế giới hạn độ cong cực đại.

Tỷ lệ thành công là $P = 100 n_{\text{đạt}}/n_{\text{thử}}$. Một lượt đạt phải tới vùng đích trong thời hạn và không va chạm trên toàn đường cuối cùng. Báo cáo trung bình, độ lệch chuẩn trên các lượt thành công và tỷ lệ thành công trên toàn bộ lượt thử; không gán độ dài bằng 0 cho lượt thất bại.

3.4.3. Bàn luận và lựa chọn Dijkstra để triển khai trên board thực tế

Với mục tiêu xây dựng mô hình xe ba bánh chạy trên bản đồ tĩnh, quy mô nhỏ và đã biết trước, đề tài lựa chọn **Dijkstra làm thuật toán tìm đường toàn cục để triển khai trên board ESP32-C3-MINI**. Quyết định này dựa trên sự phù hợp với phạm vi thực nghiệm và khả năng kiểm chứng từng bước; không xem Dijkstra là phương pháp nhanh nhất cho mọi bản đồ.

Dễ lập trình và chuyển từ mô phỏng sang phần cứng. Dijkstra gồm các thao tác rõ ràng: lấy ô có chi phí nhỏ nhất, cập nhật láng giềng và truy vết đường. Bản đồ có thể lưu bằng mảng một chiều; mỗi ô có chi phí, chỉ số ô cha và cờ trạng thái. Cấu trúc này thuận tiện chuyển từ Python sang C/C++, quan sát biến khi chạy thử và kiểm tra từng đoạn đường trên mô hình.

Ít tham số và thuận tiện thực nghiệm lặp lại. Thuật toán không cần hàm heuristic như A*, số lượng mẫu và bán kính kết nối như PRM/RRT*, hoặc hệ số hút–đẩy như APF. Khi bản đồ, trọng số, thứ tự xét láng giềng và quy tắc xử lý đồng hạng được cố định, kết quả có tính lặp lại. Nhờ đó, có thể dùng cùng tuyến tham chiếu để khảo sát ảnh hưởng của PID, sai số encoder và tốc độ chạy xe.

Có nghiệm đối chứng rõ ràng trên lưới. Với trọng số cạnh không âm, Dijkstra tìm đường có tổng chi phí nhỏ nhất khi đích được lấy ra hợp lệ; nếu hàng đợi rỗng thì không có đường trên đồ thị đã xây dựng. Đặc điểm này giúp phân biệt lỗi bản đồ, lỗi tìm đường và lỗi bám đường. Khi dùng trọng số độ dài, kết quả còn là mốc kiểm tra chi phí của A* trên cùng lưới [3].

Thuận tiện quản lý tài nguyên và tích hợp điều khiển. Với lưới nhỏ, số ô và các mảng trạng thái được biết trước, có thể cấp phát bộ nhớ cố định và sinh láng giềng khi cần thay vì lưu toàn bộ cạnh. Quy hoạch thực hiện khi nhận nhiệm vụ hoặc khi bản đồ thay đổi; chuỗi điểm sau xử lý được dùng làm tham chiếu cho bộ bám. Cách tổ chức này giúp tách việc tìm tuyến khỏi vòng đọc encoder, PID tốc độ và xuất PWM, nhưng vẫn phải đo RAM và độ trễ thực tế trên board.

Hạn chế chính là Dijkstra có thể mở rộng nhiều ô hơn A* và tăng chi phí khi lưới mịn. Bản thân nó chưa xét hướng xe, bán kính quay hay giới hạn vận tốc; đường ngắn nhất trên lưới chưa chắc là tuyến robot đi nhanh nhất hoặc dễ bám nhất. Vì vậy, đầu ra cần được rút gọn, làm mượt bằng Bézier trong vùng an toàn, kiểm tra lại va chạm rồi gán profile vận tốc hình thang trước khi đưa vào bộ bám Pure Pursuit và PID hai bánh.

Việc triển khai được đánh giá theo hai bước: trước hết đối chiếu đường đi, chi phí và trạng thái không có đường giữa Colab với board; sau đó đo thời gian quy hoạch, RAM sử dụng và khả năng hoàn thành tuyến trên xe. Dijkstra là lựa chọn cơ sở phù hợp để hiện thực hóa và kiểm chứng hệ thống; nếu bản đồ mở rộng làm vượt ngân sách tài nguyên, A* là phương án nâng cấp cần được khảo sát tiếp.